
Rencontre avec l'auxiliaire

8INF887 — APPRENTISSAGE
PROFOND — HIVER 2026





Introduction brève et structurée

- Présentation :
 - Vinith NARESH PRABAHARAN, 23 ans, étudiant français en double diplôme à l'UQAC
 - 8INF887 — Apprentissage Profond — Hiver 2026
 - Implémentation et analyse d'un système de Federated Learning pour la classification d'images

Contexte

- Deep learning classique → données centralisées
- Problème de confidentialité

→ Comment entraîner un modèle sans partager les données ?

→ Federated Learning



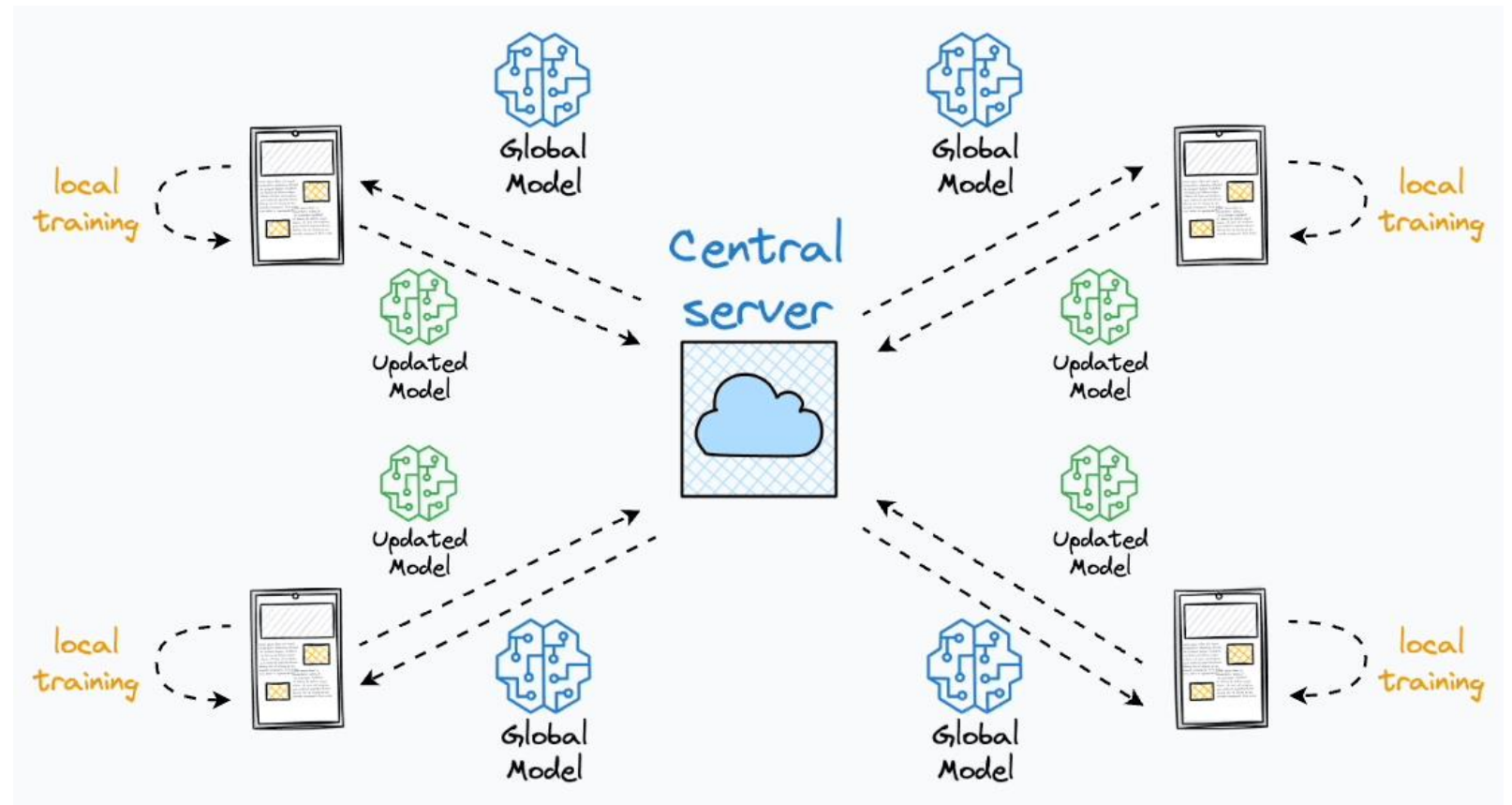
Solution

- Données restent locales
- Seuls les paramètres sont partagés

« On envoie le modèle aux données, pas les données au modèle »

Fonctionnement

1. Modèle global
2. Envoi aux clients
3. Entraînement local
4. Retour des poids
5. Agrégation



Implémentation

Le Federated Learning fonctionne en quatre étapes qui se bouclent :

1. Le serveur envoie le modèle aux clients
2. Chaque client l'entraîne localement
3. Les clients renvoient leurs modèles
4. Le serveur les agrège pour mettre à jour le modèle global

```
import copy
import torch

def federated_learning(global_model, clients_data, rounds=5,
                       local_epochs=1):
    for r in range(rounds):
        local_models = []

        # Envoi du modèle global à chaque client
        for client_loader in clients_data:
            local_model = copy.deepcopy(global_model)

            # Entraînement local
            optimizer = torch.optim.SGD(local_model.parameters(), lr=0.01)
            loss_fn = torch.nn.CrossEntropyLoss()

            local_model.train()
            for _ in range(local_epochs):
                for x, y in client_loader:
                    optimizer.zero_grad()
                    preds = local_model(x)
                    loss = loss_fn(preds, y)
                    loss.backward()
                    optimizer.step()

            local_models.append(local_model)

        # Agrégation des modèles (Federated Averaging)
        global_weights = global_model.state_dict()
        for key in global_weights.keys():
            global_weights[key] = torch.mean(
                torch.stack([model.state_dict()[key] for model in
                             local_models]),
                dim=0
            )

        global_model.load_state_dict(global_weights)

    return global_model
```

Envoi du modèle

```
# Envoi du modèle global à chaque client  
for client_loader in clients_data:  
    local_model = copy.deepcopy(global_model)
```

- Envoi du modèle global aux clients
 - Copie indépendante du modèle
- pour que l'apprentissage reste cohérent.

Entraînement local

```
# Entraînement local
optimizer = torch.optim.SGD(local_model.parameters(), lr
                             =0.01)
loss_fn = torch.nn.CrossEntropyLoss()

local_model.train()
for _ in range(local_epochs):
    for x, y in client_loader:
        optimizer.zero_grad()
        preds = local_model(x)
        loss = loss_fn(preds, y)
        loss.backward()
        optimizer.step()
```

- Entraînement local
- Optimisation SGD + loss CrossEntropy
- Backpropagation

→ diversité dans les modèles

Retour des modèles

```
local_models.append(local_model)
```

- Collecte des modèles
- Centralisation côté serveur

→ récupérer toutes les contributions des clients pour les combiner ensuite



Fusion

```
# Agrégation des modèles (Federated Averaging)
global_weights = global_model.state_dict()
for key in global_weights.keys():
    global_weights[key] = torch.mean(
        torch.stack([model.state_dict()[key] for model in
                      local_models]),
        dim=0
    )
global_model.load_state_dict(global_weights)
```

- Agrégation des modèles
 - Moyenne des paramètres
- construire un modèle global plus général



Limites et risques

- Données non-IID
- Communication
- Sécurité
- Données non identifiées



Plan d'implémentation

- Simulation de plusieurs clients
- Entraînement local (CNN)
- Implémentation de Federated Averaging
- Comparaison centralisé vs fédéré
- Expériences IID vs non-IID



Objectifs du projet

- Comprendre le Federated Learning
- Implémenter un système complet
- Analyser les performances
- Étudier l'impact des données